LEARN BEING FORWARD

Programming — Python Programming

File handling & standard library

COMPLETE LEARNING GUIDE

What You'll Learn

Real programs read configuration, write logs, parse structured data files, and handle the inevitable error cases that come with touching the outside world — Python's standard library covers all of this without needing a single third-party package. This sub-module covers safe file handling with the with statement (context managers), reading and writing both plain text and structured formats (CSV and JSON), modern path manipulation with pathlib (which has mostly replaced the older os.path string-joining style), structured exception handling for the errors file and network operations inevitably raise, and a tour of a few other standard library modules (datetime, collections, itertools) that solve extremely common problems without installing anything extra. The unifying theme is that Python's 'batteries included' philosophy means most everyday tasks have a correct, idiomatic standard-library tool already — reaching for it is almost always better than reinventing it.

1. Opening Files Safely With the with Statement

open(path, mode) returns a file object, but a file opened manually must also be closed manually (file.close()) or the underlying OS resource may leak, and if an exception is raised before the close() call, it never runs at all. The with statement wraps file handling in a context manager that guarantees close() runs automatically when the block exits, even if an exception occurs inside it — this is the reason `with open(...) as f:` is the only idiomatic way to open a file in modern Python, never a bare open()/close() pair.

- Mode strings: 'r' read (default), 'w' write (truncates existing content!), 'a' append, 'x' exclusive create (fails if the file exists), each combinable with 'b' for binary mode.
- 'w' mode silently erases an existing file's contents before writing — a very common and destructive mistake when 'a' (append) was actually intended.

PYTHON

```
with open("notes.txt", "w") as f:
    f.write("First line\n")
    f.write("Second line\n")
# file is automatically closed here, even if an exception had occurred above
```

2. Reading and Writing Text Files

Once opened, a file object can be read in full with .read(), line by line by iterating directly over it (for line in f:, the most memory-efficient option for large files since it doesn't load the whole file at once), or as a list of lines with .readlines(). Each line read this way retains its trailing newline character, which is why stripping it with .strip() is almost always needed before using the line's content.

```python
with open("notes.txt", "r") as f:
    for line in f:                # memory-efficient: one line at a time
        print(line.strip())       # strip() removes the trailing '\n'

with open("notes.txt", "r") as f:
    full_text = f.read()          # loads the entire file into memory at once
```

3. Working With CSV Files

The csv module reads and writes comma-separated (or other delimiter-separated) tabular data correctly, including edge cases like commas or quotes embedded inside a field, that a naive line.split(',') approach would mishandle. csv.DictReader reads each row directly into a dict keyed by the header row's column names, which is almost always more convenient than working with plain index-based lists of strings.

```python
import csv

with open("people.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "age"])
    writer.writeheader()
    writer.writerow({"name": "Asha", "age": 29})
    writer.writerow({"name": "Ben", "age": 34})

with open("people.csv", "r", newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], row["age"])  # values come back as strings
```

4. Working With JSON

The json module converts between Python objects and JSON text: json.dumps(obj) serializes a Python dict/list into a JSON string (and json.dump(obj, file) writes it directly to a file), while json.loads(text) parses a JSON string back into Python objects (json.load(file) reads directly from a file). JSON's type system is narrower than Python's — tuples become JSON arrays (indistinguishable from lists once round-tripped), and dict keys are always coerced to strings on the way out, which can silently change integer keys into string keys.

- json.dumps(data, indent=2) produces human-readable, pretty-printed output — useful for config files and debugging, unnecessary for machine-to-machine payloads.
- Round-tripping a dict with integer keys through JSON turns those keys into strings, since JSON object keys are always strings.

```python
import json

config = {"name": "MyApp", "version": 2, "features": ["dark_mode", "export"]}

with open("config.json", "w") as f:
    json.dump(config, f, indent=2)

with open("config.json", "r") as f:
    loaded = json.load(f)

print(loaded["features"])  # ['dark_mode', 'export']
```

5. Modern Path Handling With pathlib

pathlib.Path represents a filesystem path as an object with methods and overloaded operators, replacing the older style of joining path strings manually with os.path.join. The / operator joins path segments in a way that's automatically correct for the current OS (backslashes on Windows, forward slashes elsewhere), and methods like .exists(), .is_file(), .mkdir(), and .glob() cover the overwhelming majority of filesystem tasks without needing the older os module's scattered functions.

```python
from pathlib import Path

data_dir = Path("data") / "reports"
data_dir.mkdir(parents=True, exist_ok=True)

report_file = data_dir / "summary.txt"
report_file.write_text("Q3 results look strong.\n")

for csv_file in data_dir.glob("*.csv"):
    print(csv_file.name)

print(report_file.exists(), report_file.suffix)  # True .txt
```

6. Exception Handling for File and I/O Errors

File and network operations fail in ways that are entirely normal, not exceptional edge cases — a file might not exist (FileNotFoundError), a path might be a directory instead of a file (IsADirectoryError), or permissions might be insufficient (PermissionError). try/except should catch the specific exception types expected, not a bare except: (which also silently swallows genuinely unexpected bugs like a NameError), and an optional finally block runs cleanup code whether or not an exception occurred.

• Catching a bare `except:` (or even `except Exception:` too broadly) can hide real bugs — catch the specific exception type(s) you actually expect and can meaningfully handle.

• Custom exceptions (class MyError(Exception): pass) let application code raise and catch domain-specific errors distinctly from built-in ones.

```python
def read_config(path: str) -> dict:
    try:
        with open(path, "r") as f:
            return json.load(f)
    except FileNotFoundError:
        print(f"Config file not found at {path}, using defaults")
        return {}
    except json.JSONDecodeError as e:
        raise ValueError(f"Config file at {path} is not valid JSON") from e
```

7. A Tour of Essential Standard Library Modules

Beyond file I/O, a handful of standard library modules solve recurring problems so commonly that reaching for them should be automatic. datetime provides date/time arithmetic and formatting (datetime.now(), timedelta for adding/subtracting durations). collections.Counter tallies item frequencies in one line; collections.defaultdict avoids manual 'if key not in dict' initialization checks. itertools provides efficient, composable iterator building blocks (chain, combinations, groupby) for problems that would otherwise need manual nested loops.

```python
from datetime import datetime, timedelta
from collections import Counter, defaultdict

now = datetime.now()
in_a_week = now + timedelta(days=7)

words = ["cat", "dog", "cat", "bird", "dog", "cat"]
print(Counter(words))  # Counter({'cat': 3, 'dog': 2, 'bird': 1})

groups = defaultdict(list)
for word in words:
    groups[len(word)].append(word)
print(dict(groups))   # {3: ['cat','dog','cat','dog','cat'], 4: ['bird']}
```

Common Pitfalls

• Opening a file with a bare open()/close() pair instead of `with open(...) as f:`, risking a leaked file handle if an exception occurs before close().

• Opening an existing file in 'w' mode when 'a' (append) was intended, silently truncating and losing its previous contents.

- Parsing CSV manually with line.split(',') instead of the csv module, breaking on any field containing a comma or quote character.
- Forgetting that JSON round-trips coerce dict keys to strings and turn tuples into plain arrays, then being surprised when loaded data doesn't match the original types.
- Catching a bare `except:` that swallows every error indiscriminately, including unrelated bugs that should have crashed loudly and been noticed.
- Manually joining path strings with '/' or '\\' instead of pathlib's Path, producing code that breaks when run on a different operating system.

Key Takeaways

- Always open files with `with open(...) as f:` — it guarantees the file is closed even if an exception occurs inside the block.
- 'w' mode truncates an existing file; use 'a' to append instead — a very common source of accidental data loss.
- Use the csv and json modules for structured data instead of manual string splitting — they correctly handle escaping and edge cases.
- pathlib.Path with the / operator is the modern, OS-independent way to build filesystem paths.
- Catch specific exception types (FileNotFoundError, json.JSONDecodeError) rather than a bare except:, which hides unrelated bugs.
- Standard library modules like collections (Counter, defaultdict) and itertools solve many common data-shuffling problems without any external dependency.

Learn More

Python Docs — Reading and Writing Files
https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files

Python Docs — pathlib: Object-Oriented Filesystem Paths
https://docs.python.org/3/library/pathlib.html

Python Docs — json: JSON Encoder and Decoder
https://docs.python.org/3/library/json.html